

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

2^η Σειρά Ασκήσεων



Μία αναφορά για το μάθημα

Στατιστική Μοντελοποίηση και Αναγνώριση Προτύπων

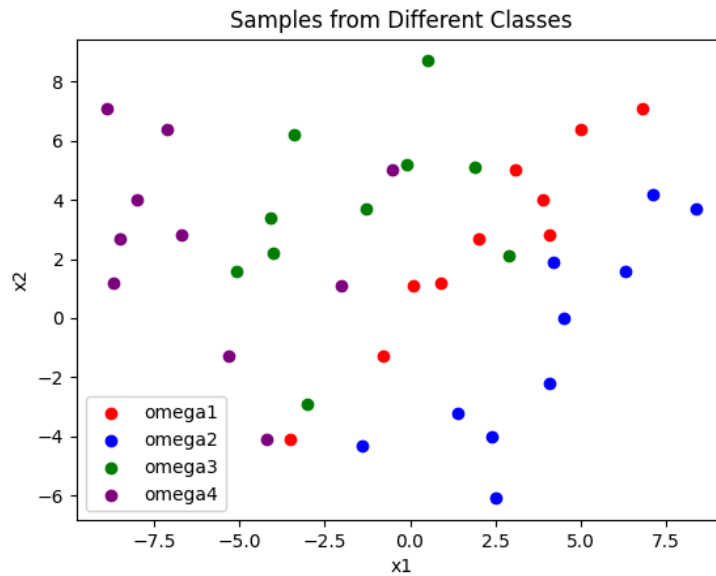
ΣΤΑΥΡΟΠΟΥΛΟΥ ΕΛΕΝΗ Α.Μ.: 2019030070

ΓΙΑΟΥΠΙ ΚΛΕΪΝΤΑ Α.Μ.: 2019030132

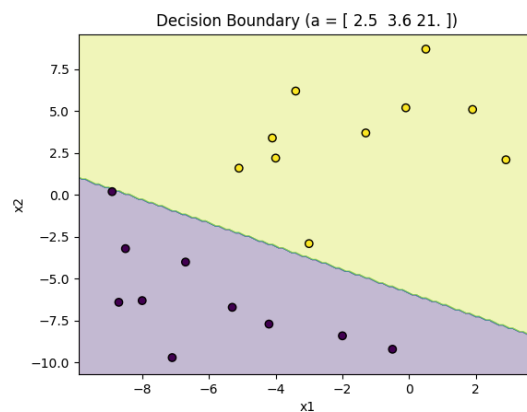
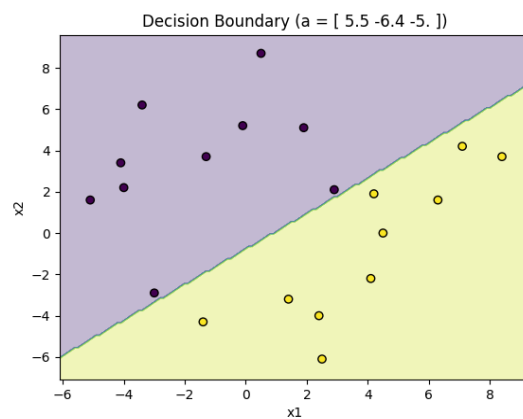
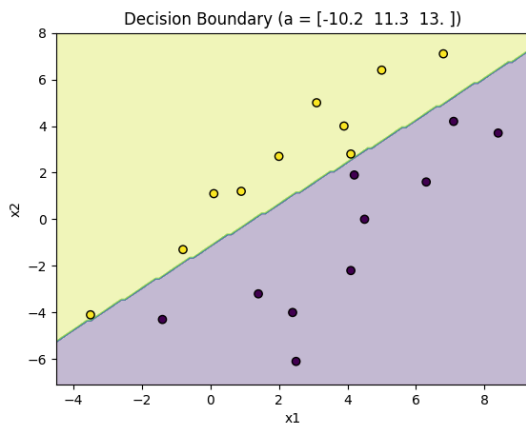
Χανιά, 2024

Θέμα 1 - Αλγόριθμος Perceptron

Αρχικά, σχεδιάστηκαν στο ίδιο διάγραμμα τα δείγματα με διαφορετικό χρώμα για κάθε κλάση.



Έπειτα, εφαρμόστηκε ο αλγόριθμος Perceptron για ταξινόμηση των δειγμάτων των κλάσεων ω_1 και ω_2 , ω_2 και ω_3 , ω_3 και ω_4 με αρχικές τιμές βαρών 0.

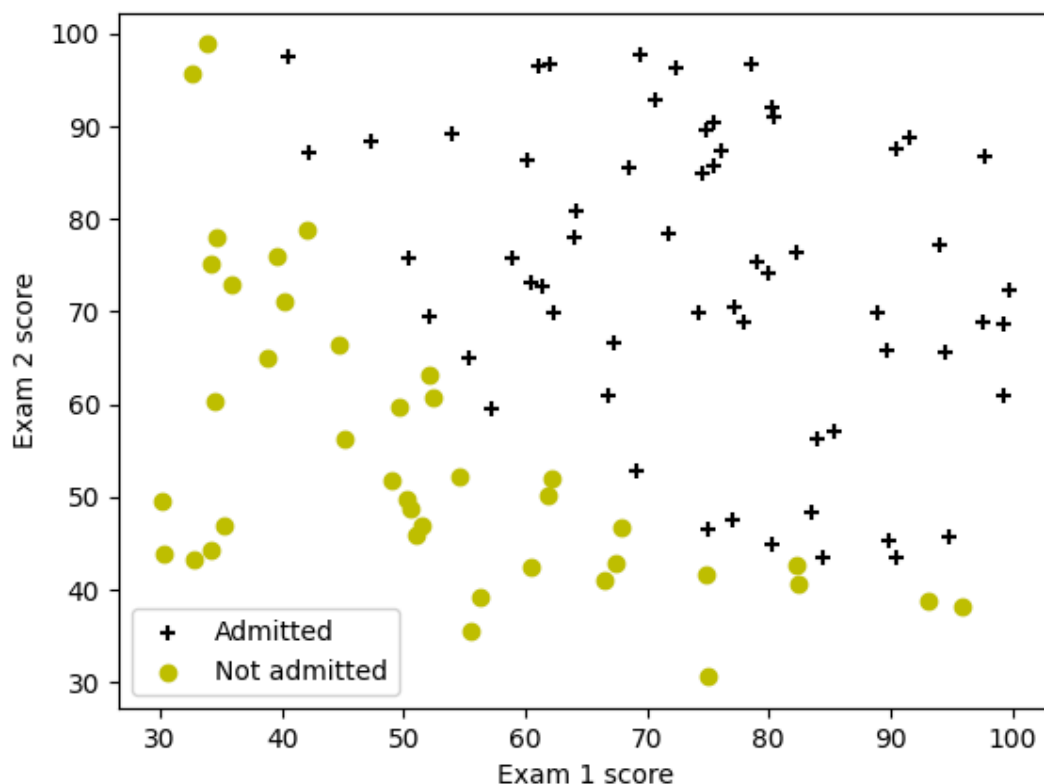


Εκτυπώθηκε στην οθόνη ο αριθμός των επαναλήψεων μέχρι να συγκλίνει ο αλγόριθμος. Παρατηρείται ότι τα γραμμικά διαχωρίσιμα δεδομένα συγκλίνουν γρηγορότερα, ενώ οι πλησιέστερες ή επικαλυπτόμενες κλάσεις χρειάζονται περισσότερο χρόνο για να συγκλίνουν.

```
Class 1 vs Class 2: Number of iterations = 9
Class 2 vs Class 3: Number of iterations = 5
Class 3 vs Class 4: Number of iterations = 23
```

Θέμα 2 - Λογιστική Παλινδρόμηση: Αναλυτική εύρεση κλίσης (Gradient)

Πρωτίστως, συμπληρώθηκε ο κώδικας python στο αρχείο MyLogit.py και έτσι διαβάστηκαν και σχεδιάσαν τα δείγματά.

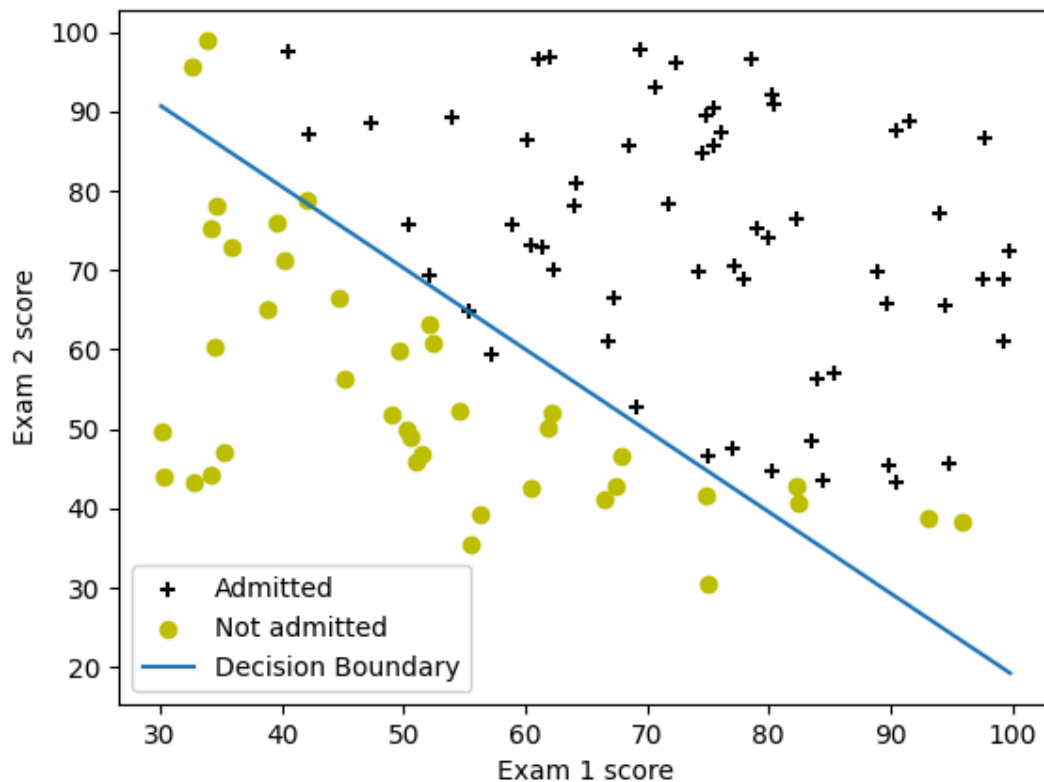


Έπειτα, επιστρέφεται το κόστος $J(\theta)$ (J) και η κλίση $\nabla J(\theta)$ (grad) όπως ορίζονται στο ερώτημα (α).

```
Cost at initial theta (zeros): 0.8213534164441316
Gradient at initial theta (zeros): [ -0.26666667 -22.94992893 -22.29984189]
```

```
Cost at theta found by minimize: 0.20349770159526337
Theta: [-24.46816127  0.20623111  0.20147182]
```

Συμπληρώθηκε ο κώδικας στην συνάρτηση predict για να προβλέψετε αν ο φοιτητής θα γίνει δεκτός με βάση διάφορες τιμές βαθμών στις δύο εξετάσεις.



Τέλος, ελέγχθηκε το αποτέλεσμα για έναν φοιτητή που έγραψε στο πρώτο μάθημα 45 και στο δεύτερο 85. Η πιθανότητα να γίνει δεκτός φαίνεται παρακάτω:

```
For a student with scores 45 and 85, we predict an admission probability of 77.63%
Train Accuracy: 89.00%
```

Θέμα 3 - Εκτίμηση Παραμέτρων με Maximum Likelihood

Υπολογίστηκαν οι τιμές μεγίστης πιθανοφάνειας $\hat{\mu}$ και $\hat{\sigma}^2$ για καθένα από τα 3 χαρακτηριστικά των κλάσεων ω_1 και ω_2 .

```
Mean ( $\hat{\mu}$ ) for each characteristic: [-0.0709 -0.6047 -0.911 ]
Variance ( $\hat{\sigma}^2$ ) for each characteristic: [0.90617729 4.20071481 4.541949 ]
```

```
 $\omega_2$  - Mean ( $\hat{\mu}$ ) for each characteristic: [-0.1126 0.4299 0.00372]
 $\omega_2$  - Variance ( $\hat{\sigma}^2$ ) for each characteristic: [0.05392584 0.04597009 0.00726551]
```

Θεωρώντας ότι ανά δύο τα χαρακτηριστικά της κάθε κλάσης ακολουθούν 2D κανονική κατανομή υπολογίστηκαν οι παράμετροι της κανονικής κατανομής με τη μέθοδο της μέγιστης πιθανοφάνειας.

```
Pair 1: Features 1 and 2
Mean vector ( $\hat{\mu}$ ): [-0.0709 -0.6047]
Covariance matrix ( $\hat{\Sigma}$ ):
[[0.90617729 0.56778177]
 [0.56778177 4.20071481]]
```

```
Pair 2: Features 1 and 3
Mean vector ( $\hat{\mu}$ ): [-0.0709 -0.911 ]
Covariance matrix ( $\hat{\Sigma}$ ):
[[0.90617729 0.3940801 ]
 [0.3940801 4.541949 ]]
```

```
Pair 3: Features 2 and 3
Mean vector ( $\hat{\mu}$ ): [-0.6047 -0.911 ]
Covariance matrix ( $\hat{\Sigma}$ ):
[[4.20071481 0.7337023 ]
 [0.7337023 4.541949 ]]
```

```
 $\omega_2$  - Pair 1: Features 1 and 2
Mean vector ( $\hat{\mu}$ ): [-0.1126 0.4299]
Covariance matrix ( $\hat{\Sigma}$ ):
[[ 0.05392584 -0.01465126]
 [-0.01465126 0.04597009]]
```

```
 $\omega_2$  - Pair 2: Features 1 and 3
Mean vector ( $\hat{\mu}$ ): [-0.1126 0.00372]
Covariance matrix ( $\hat{\Sigma}$ ):
[[ 0.05392584 -0.00517993]
 [-0.00517993 0.00726551]]
```

```
 $\omega_2$  - Pair 3: Features 2 and 3
Mean vector ( $\hat{\mu}$ ): [0.4299 0.00372]
Covariance matrix ( $\hat{\Sigma}$ ):
[[0.04597009 0.00850987]
 [0.00850987 0.00726551]]
```

Αντίστοιχοι υπολογισμοί έγιναν θεωρώντας ότι το συνολικό διάνυσμα χαρακτηριστικών της κάθε κλάσης ακολουθεί 3D κανονική κατανομή.

```
Mean vector ( $\hat{\mu}$ ) for 3D: [-0.0709 -0.6047 -0.911 ]
Covariance matrix ( $\hat{\Sigma}$ ) for 3D:
[[0.90617729 0.56778177 0.3940801 ]
 [0.56778177 4.20071481 0.7337023 ]
 [0.3940801 0.7337023 4.541949 ]]
```

```
 $\omega_2$  - Mean vector ( $\hat{\mu}$ ) for 3D: [-0.1126 0.4299 0.00372]
 $\omega_2$  - Covariance matrix ( $\hat{\Sigma}$ ) for 3D:
[[ 0.05392584 -0.01465126 -0.00517993]
 [-0.01465126 0.04597009 0.00850987]
 [-0.00517993 0.00850987 0.00726551]]
```

Τέλος, με την υπόθεση ότι το τρισδιάστατο μοντέλο είναι διαχωρίσιμο, εκτιμήθηκε με τη μέθοδο μεγίστης πιθανοφάνειας το μέσο $\hat{\mu}$ και τα διαγώνια στοιχεία του $\hat{\Sigma}$.

```
Mean vector ( $\hat{\mu}$ ) for diagonal covariance matrix: [-0.0709 -0.6047 -0.911 ]  
Diagonal elements of covariance matrix ( $\hat{\sigma}^2$ ): [0.90617729 4.20071481 4.541949 ]
```

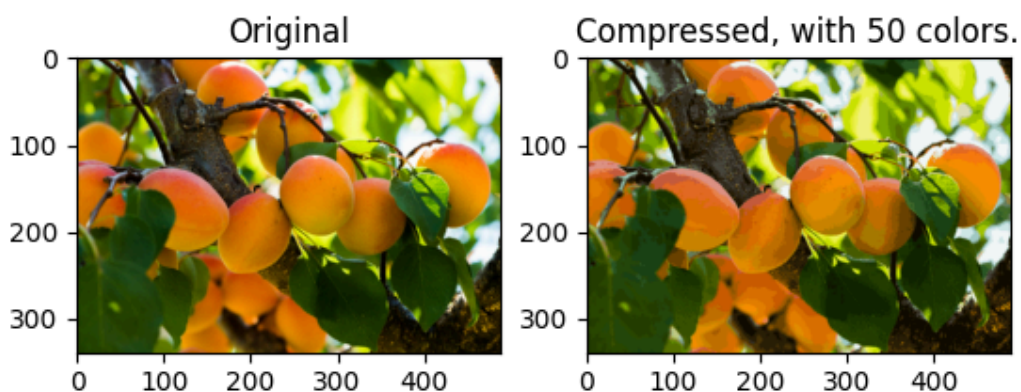
```
 $\omega_2$  - Mean vector ( $\hat{\mu}$ ) for diagonal covariance matrix: [-0.1126  0.4299  0.00372]  
 $\omega_2$  - Diagonal elements of covariance matrix ( $\hat{\sigma}^2$ ): [0.05392584 0.04597009 0.00726551]
```

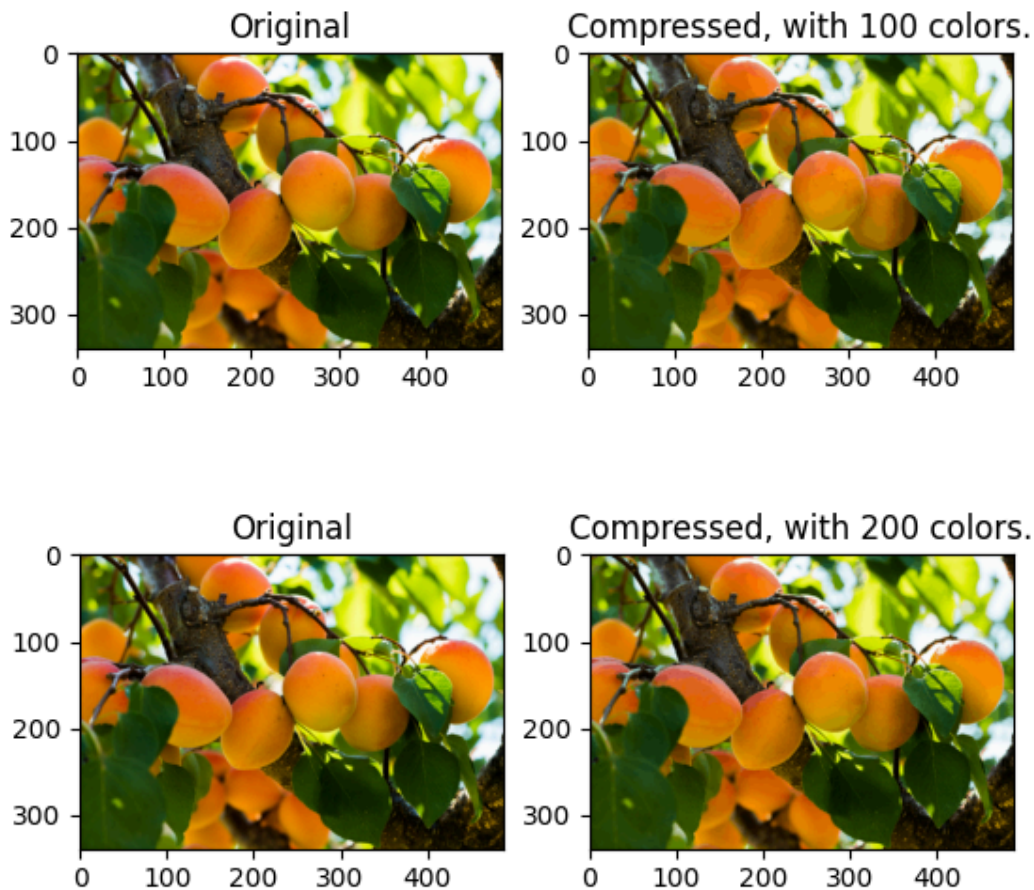
Συγκρίνοντας τα αποτελέσματα μεταξύ τους παρατηρείται ότι για το τρισδιάστατο μοντέλο τα $\hat{\mu}$ και $\hat{\Sigma}$ έχουν ίδιες τιμές με το δισδιάστατο και υπάρχει μία επιπλέον τιμή για το $\hat{\mu}$ και μία επιπλέον γραμμή και μία στήλη για το $\hat{\Sigma}$.

Οι διαφορές ανάμεσα στις 2 κλάσεις προκύπτουν επειδή τα δύο σύνολα δεδομένων (αρχικό και ω_2) έχουν διαφορετικές κατανομές, κεντρικές τάσεις και διασπορά. Οι διαφορές αυτές μπορεί να οφείλονται σε διαφορετικές μεθόδους δειγματοληψίας, προεπεξεργασία δεδομένων ή εγγενή χαρακτηριστικά των συνόλων δεδομένων.

Θέμα 4 - Ομαδοποίηση (Clustering) με K-means και GMM

Πρωτίστως, συμπληρώθηκε ο κώδικας στο αρχείο myKmeans.py και έπειτα χρησιμοποιώντας την εικόνα 'Fruit.png', την οποία εμφανίσαμε, συγκρίναμε με την συμπίεσμένη εικόνα. Τέλος, δοκιμάσαμε να τρέξουμε τον αλγόριθμο για διαφορετικό αριθμό κλάσεων/χρωμάτων K (50, 100 και 200).





Παρατηρήσαμε ότι για:

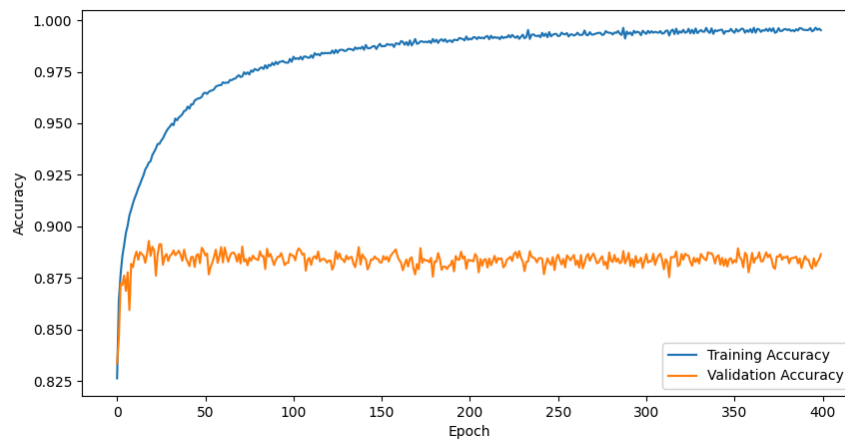
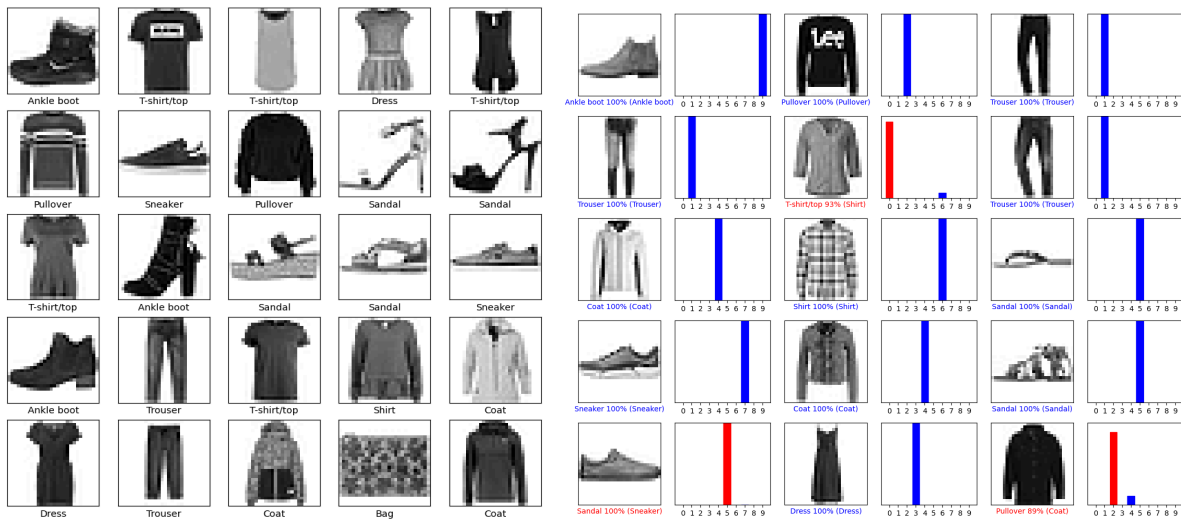
Χαμηλότερες τιμές K (50): Αυτές έχουν ως αποτέλεσμα υψηλότερους λόγους συμπίεσης αλλά σημαντικά χαμηλότερη ποιότητα εικόνας, με σημαντική απώλεια λεπτομερειών και χρωματικών πληροφοριών. Αυτές οι ρυθμίσεις είναι χρήσιμες όταν απαιτείται ακραία συμπίεση και η ποιότητα είναι λιγότερο σημαντική.

Μεσαίες τιμές K (100): Αυτές παρέχουν μια ισορροπία μεταξύ συμπίεσης και ποιότητας εικόνας. Οι εικόνες διατηρούν τις απαραίτητες λεπτομέρειες και χρώματα, καθιστώντας αυτές τις ρυθμίσεις κατάλληλες για γενική χρήση όπου τόσο η ποιότητα όσο και η συμπίεση είναι σημαντικές.

Υψηλότερες τιμές K (200): Αυτές οδηγούν σε εικόνες πολύ υψηλής ποιότητας, η οποία είναι πολύ κοντά στην αρχική, αλλά το μέγεθος των δεδομένων είναι μεγαλύτερο σε σύγκριση με τις χαμηλότερες τιμές K . Αυτές οι ρυθμίσεις είναι κατάλληλες όταν η υψηλή ποιότητα είναι προτεραιότητα και το εύρος ζώνης αποθήκευσης ή μετάδοσης δεν αποτελεί περιοριστικό παράγοντα.

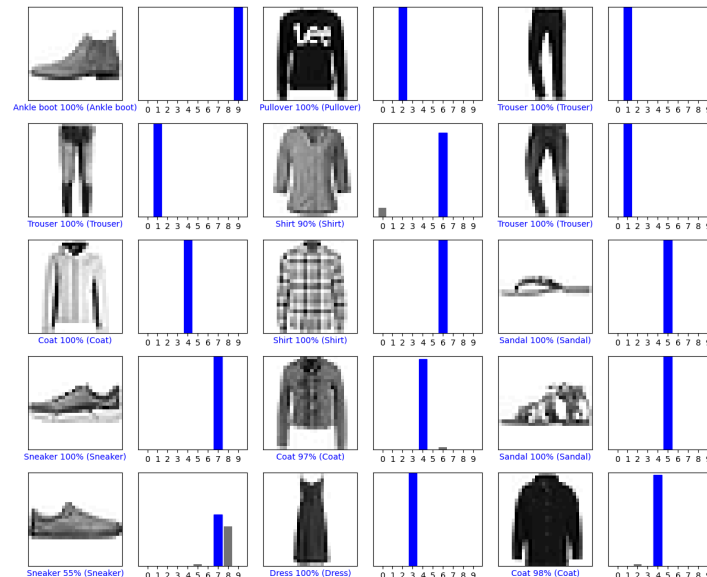
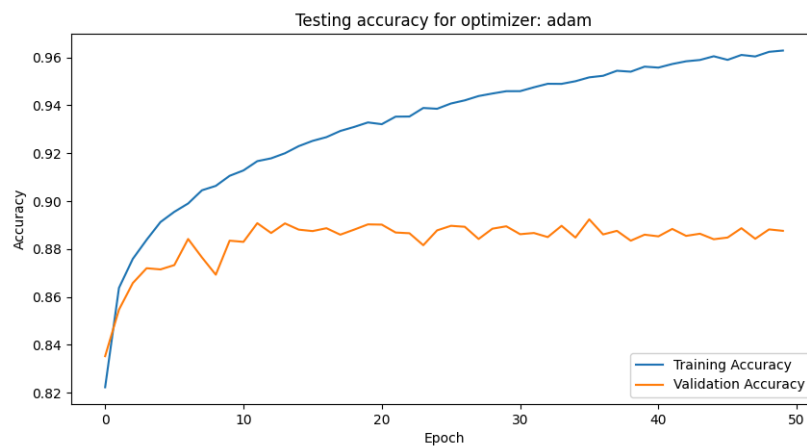
Θέμα 5β - Convolutional Neural Networks for Image Recognition

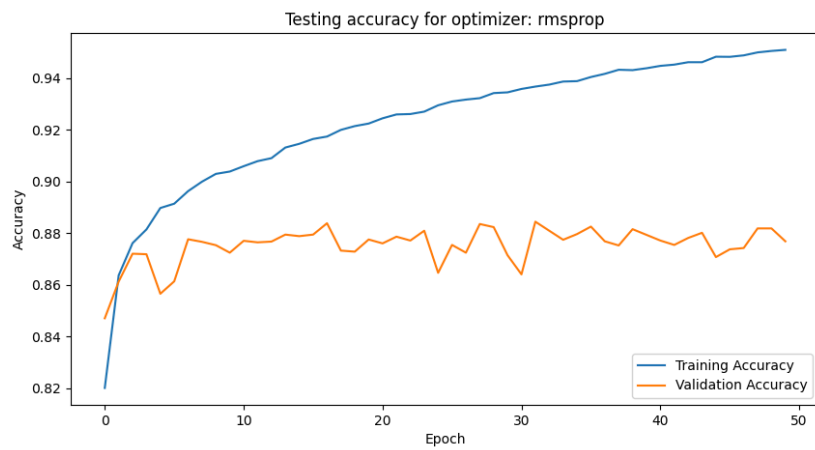
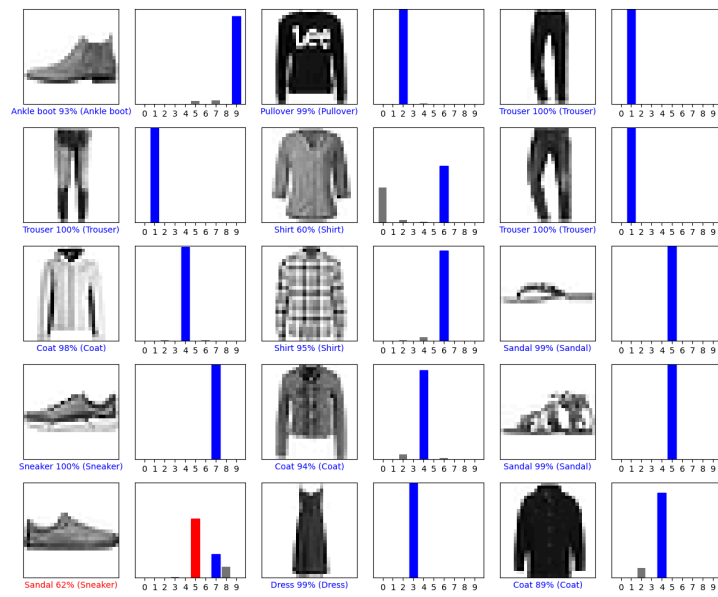
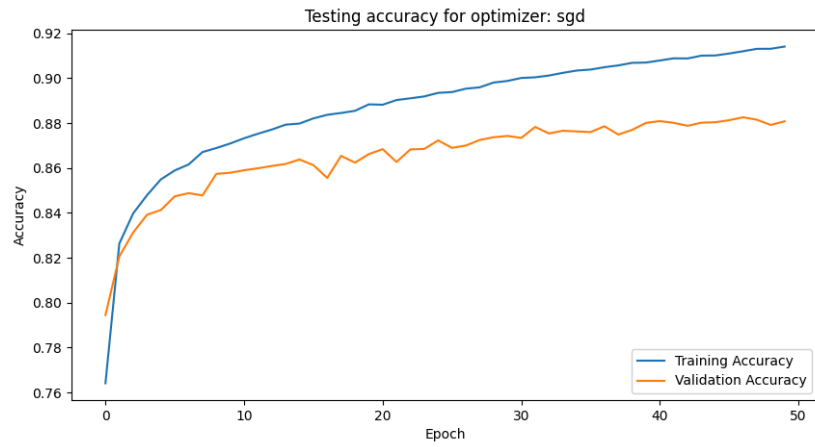
- 1) Αρχικά τρέξαμε το πρόγραμμα με την αρχιτεκτονική που δίνεται και με epoch = 400. Παρατηρήσαμε ότι το training accuracy αυξήθηκε πολύ γρήγορα και έφτασε σχεδόν το 100% μέσα σε 100-150 epochs και αντίστοιχα το validation accuracy έφτασε σε λιγότερα από 50 epochs σχεδόν το 90%. Αυτό σημαίνει ότι το μοντέλο είναι αρκετά πολύπλοκο ώστε να ταιριάζει απόλυτα στα δεδομένα εκπαίδευσης. Επίσης, κάνει καλή γενίκευση σε άγνωστα δεδομένα και δεν υπάρχει overfitting.

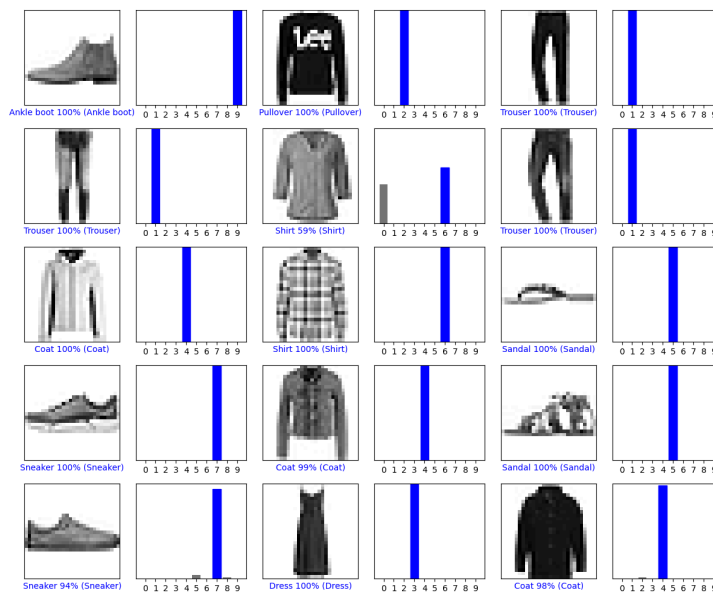
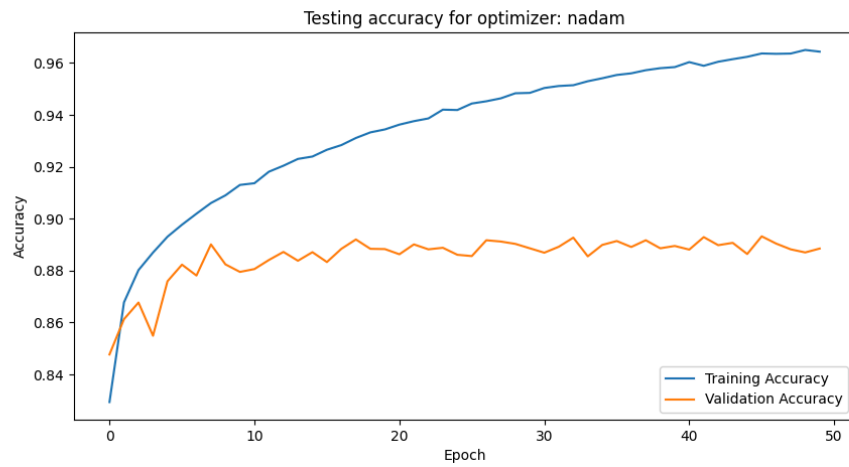
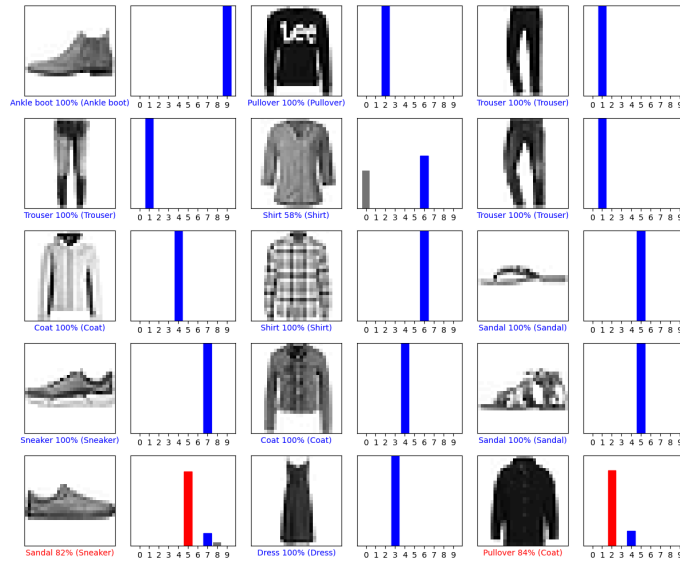


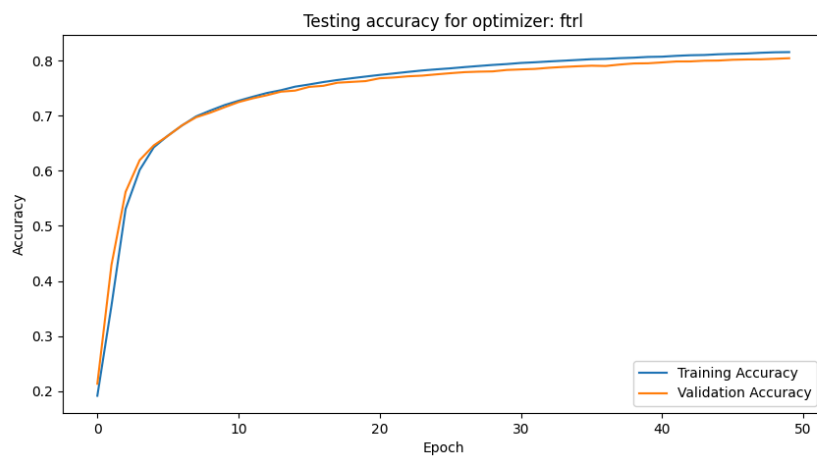
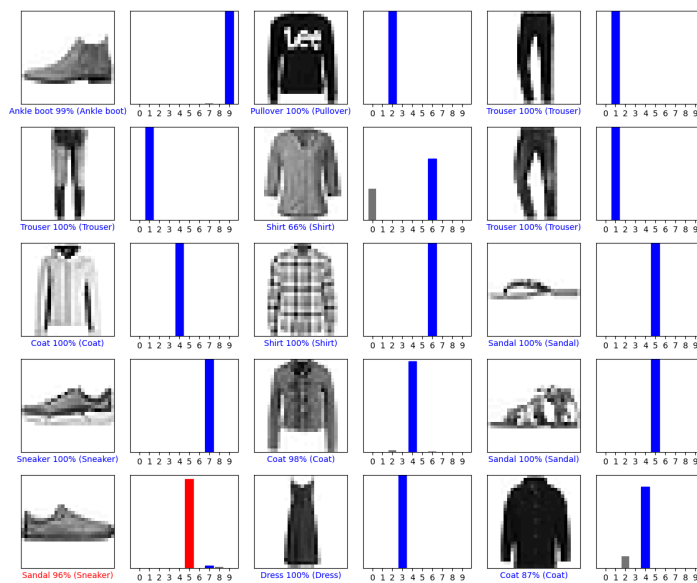
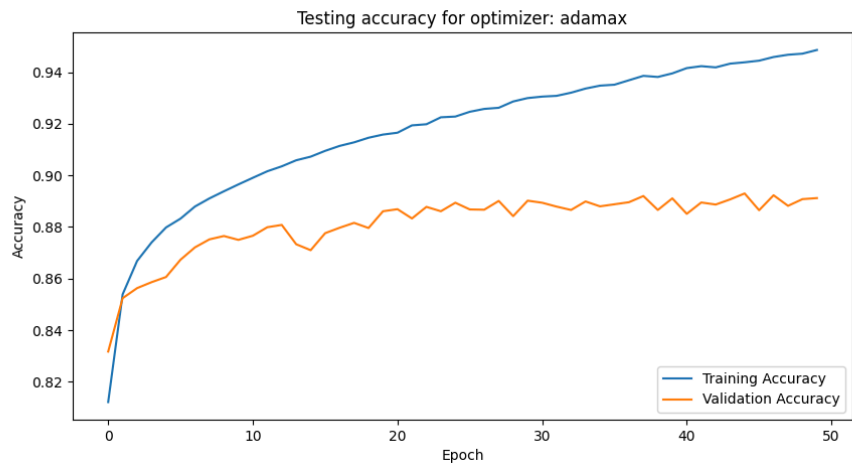
- 2) Στη συνέχεια, δοκιμάσαμε τους διάφορους αλγορίθμους βελτιστοποίησης και συγκρίναμε τα αποτελέσματα ώστε να βρούμε τον βέλτιστο. Όπως φαίνεται, το καλύτερο test accuracy επιτυγχάνεται με τον αλγόριθμο adamax, ο οποίος χειρίζεται καλά τις μεγάλες κλίσεις, προσαρμόζει αποτελεσματικά τους ρυθμούς μάθησης και παρέχει σταθερή εκπαίδευση, οπότε είναι κατάλληλος για dataset Fashion-MNIST.

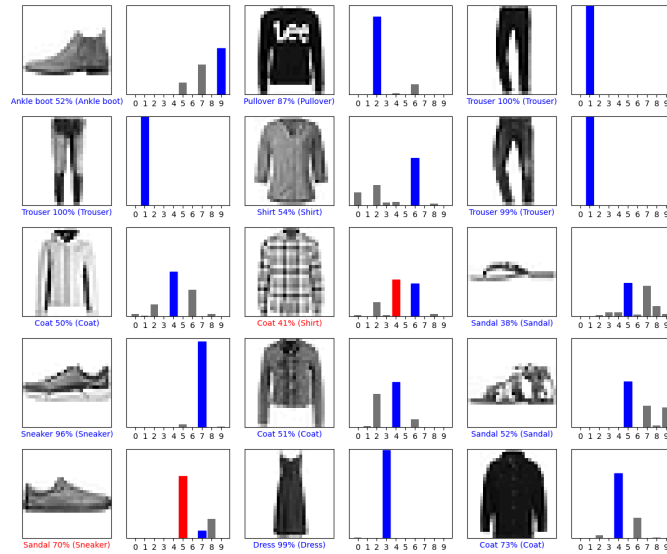
```
Optimizer: sgd, Test accuracy: 0.880800008773803730042
Optimizer: rmsprop, Test accuracy: 0.87690000172615051
Optimizer: nadam, Test accuracy: 0.8884999752044678
Optimizer: adamax, Test accuracy: 0.8912000060081482
Optimizer: ftrl, Test accuracy: 0.8043000102043152
```



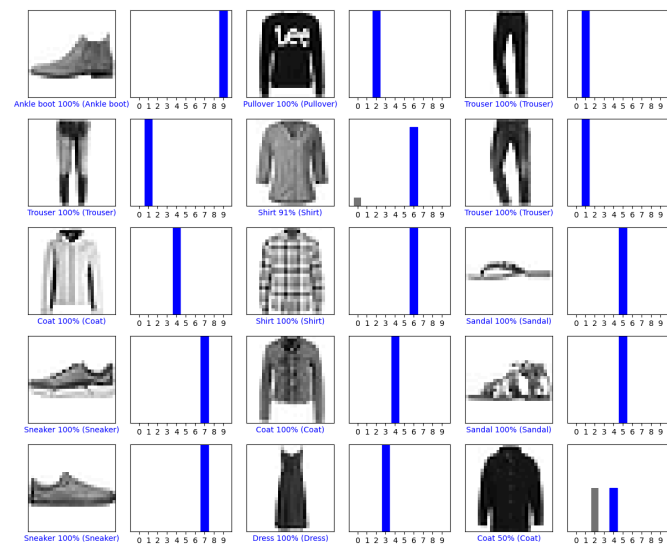
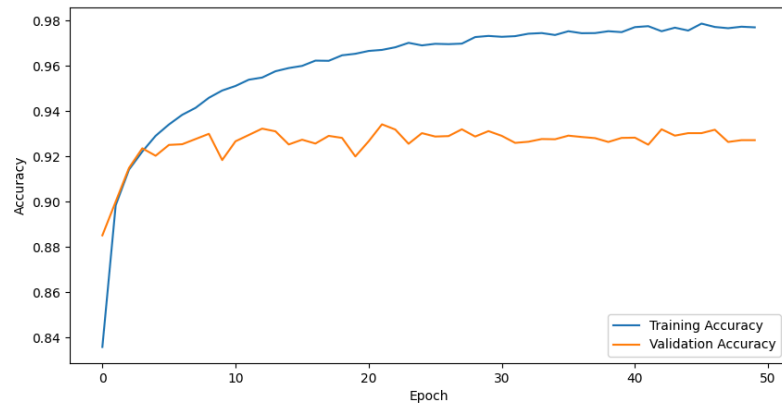




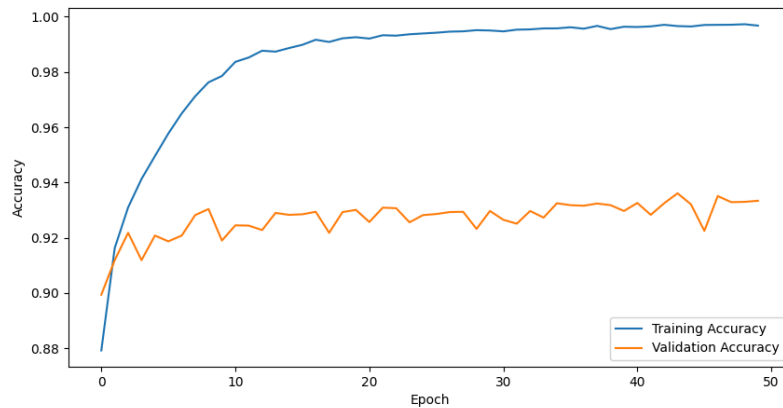




- 3) Αφού έγινε η αλλαγή στην αρχιτεκτονική του δικτύου, παρατηρήθηκε βελτίωση τόσο στο training όσο και στο validation accuracy, γεγονός που οφείλεται στο ότι τα convolution layers είναι καλύτερα στην εξαγωγή των χαρακτηριστικών από τις εικόνες.



- 4) Έπειτα, η προσθήκη του Batch Normalization πριν την συνάρτηση ReLU ομαλοποίησε τις εισόδους σε κάθε επίπεδο, οπότε παρατηρήθηκε ταχύτερη και σταθερότερη εκπαίδευση.



- 5) Τέλος, με το Dropout ενισχύθηκε το validation accuracy, αλλά η εκπαίδευση ήταν ελαφρώς πιο αργή και λιγότερο ακριβής.

